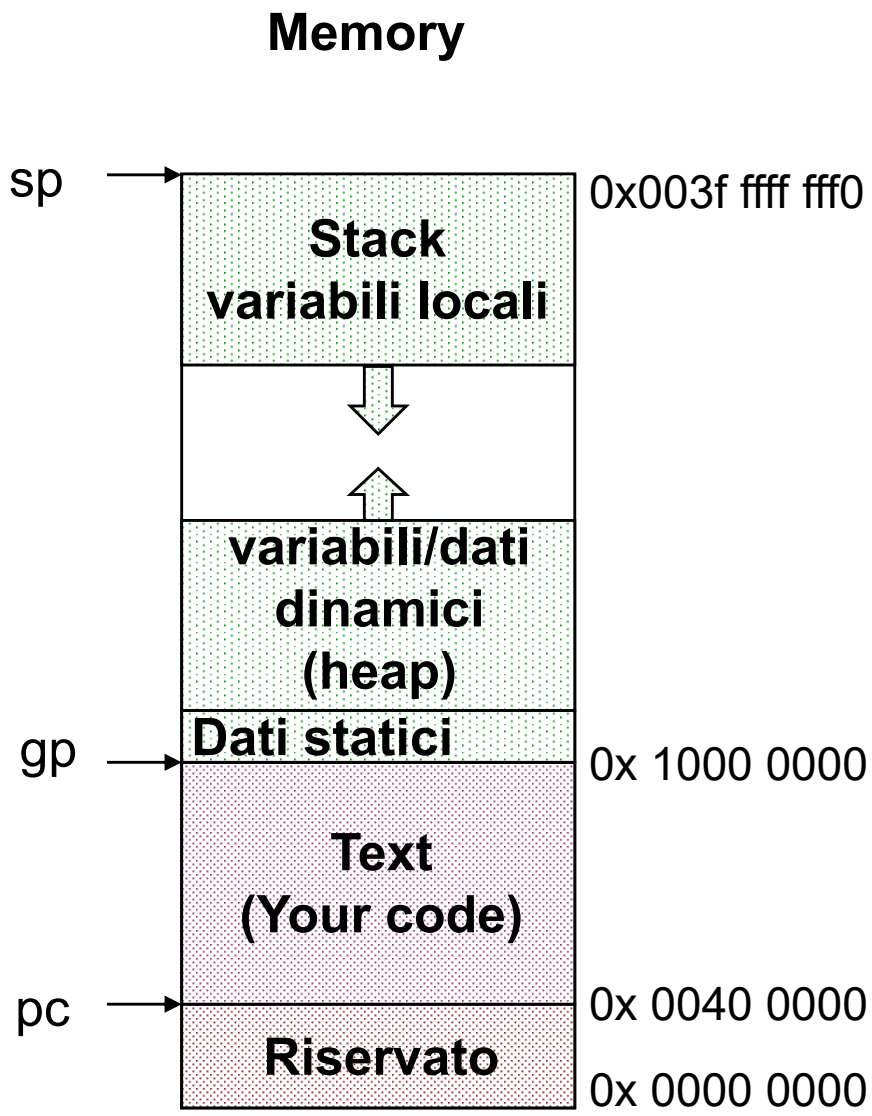
Simulatore RISC-V
QtRvSim

Modello della memoria RISC-V in C/C++



Un semplice esempio di programma

```
# somma di due numeri interi


.text                          # sezione text
.globl main                    # chiamata procedura main


main:   la t0, valore          # load 'valore' in t0
        lw t1, 0(t0)           # load word 0(valore) in t1
        lw t2, 4(t0)           # load word 4(valore) in t2
        add t3, t1, t2         # somma due numeri in t3
        sw t3, 8(t0)           # store word t3 in 8(t0)
        li a0, 10              # mettere in a0 il numero «10»
        ecall                  # chiamata a sistema
.data                          # sezione dati
valore: .word 10, 20, 0        # inizializzazione 'valore'
```

Direttive

- **.text**

  indicates that following items are stored in the user text segment, typically instructions

- **.data**

  indicates that following data items are stored in the data segment

- **.globl** sym

  declare that symbol sym is global and can be referenced from other files

(*) presente nell'ISA RISC-V, ma non in QtRvSim

- **.word** w1, …, wn

  store n 32-bit quantities in successive memory words

- **.half** h1, …, hn

  store n 16-bit quantities in successive memory halfwords (*)

- **.byte** b1, …, bn

  store n 8-bit quantities in successive memory bytes

- **.ascii** str

  store the string in memory but do not null-terminate it

  strings are represented in double-quotes "str"
  special characters, eg. \n, \t, follow C convention

- **.asciz** str

  alias for **.string**

- **.string** str

  store the string in memory and null-terminate it (*)

- **.float** f1, …, fn

  store n floating point single precision numbers in successive memory locations (*)

- **.double** d1, …, dn

  store n floating point double precision numbers in successive memory locations (*)

Un secondo esempio: scambio di 2 parole in memoria

```
## swap di due parole

.text
.globl main

main:
la      s0, data_1 # load data of data_1
lw      s1, 0(s0)
lw      s2, 4(s0)
sw      s2, 0(s0)
sw      s1, 4(s0)

li      a0, 10
ecall

.data               # load data
data_1: .word 7
data_2: .word 3
```

Template per la scrittura del codice

```
# ========== Data Segment
.data
# inserire qui inizializzazione dati

# ========== Code Segment
.text
.globl  main
main:
# inserire qui il codice assembler

# richiesta di terminare l'esecuzione del
programma
EXIT:
li a0,10
ecall

# end of file
```

Chiamata a sistema

- System Calls (ecall)
  - Invocazione servizi del sistema operativo (emulati)
- Funzionamento
  - Caricare il codice della chiamata nel registro a0
  - Caricare gli argomenti nei registri a1…a7
  - Invocare l'esecuzione con l'istruzione `ecall`
  - Dopo l'esecuzione della chiamata l'eventuale valore di ritorno è nei registri a0 e a1
- Principali chiamate a sistema (in RISC-V)

servizio	codice (a0)	argomenti	risultato
print_int	1	a1	
print_string	4	a1	
print_characters	11	a1	
read_str	0x130		a1
read_char	0x131		a1
exit	10		

Chiamata a sistema

- Nel simulatore QtRvSim le chiamate a sistema sono riferite al sistema operativo Linux.
- Infatti si usa come input e output il terminale fornito dal simulatore stesso.
- Sono supportate diverse chiamate a sistema*, ma qui vengono riportate la lettura da e la stampa sul terminale integrato

Servizio	Codice (a7)	File descriptor (a0)	Buffer (a1)	Count (a2)	Risultato (a0)
read	63	1	Puntatore all'elemento da sovrascrivere con lettura input	N. byte da leggere	N. di byte letti**
write	64	1	Puntatore all'elemento da stampare in output	N. byte da stampare	N. di byte stampati
Servizio	Codice (a7)	exit status (a0)			
exit	93	….			

- Altre chiamate sono indicate al seguente link

* https://github.com/cvut/qtrvsim?tab=readme-ov-file#system-calls-support

** + 1, perché viene aggiunto automaticamente il carattere newline

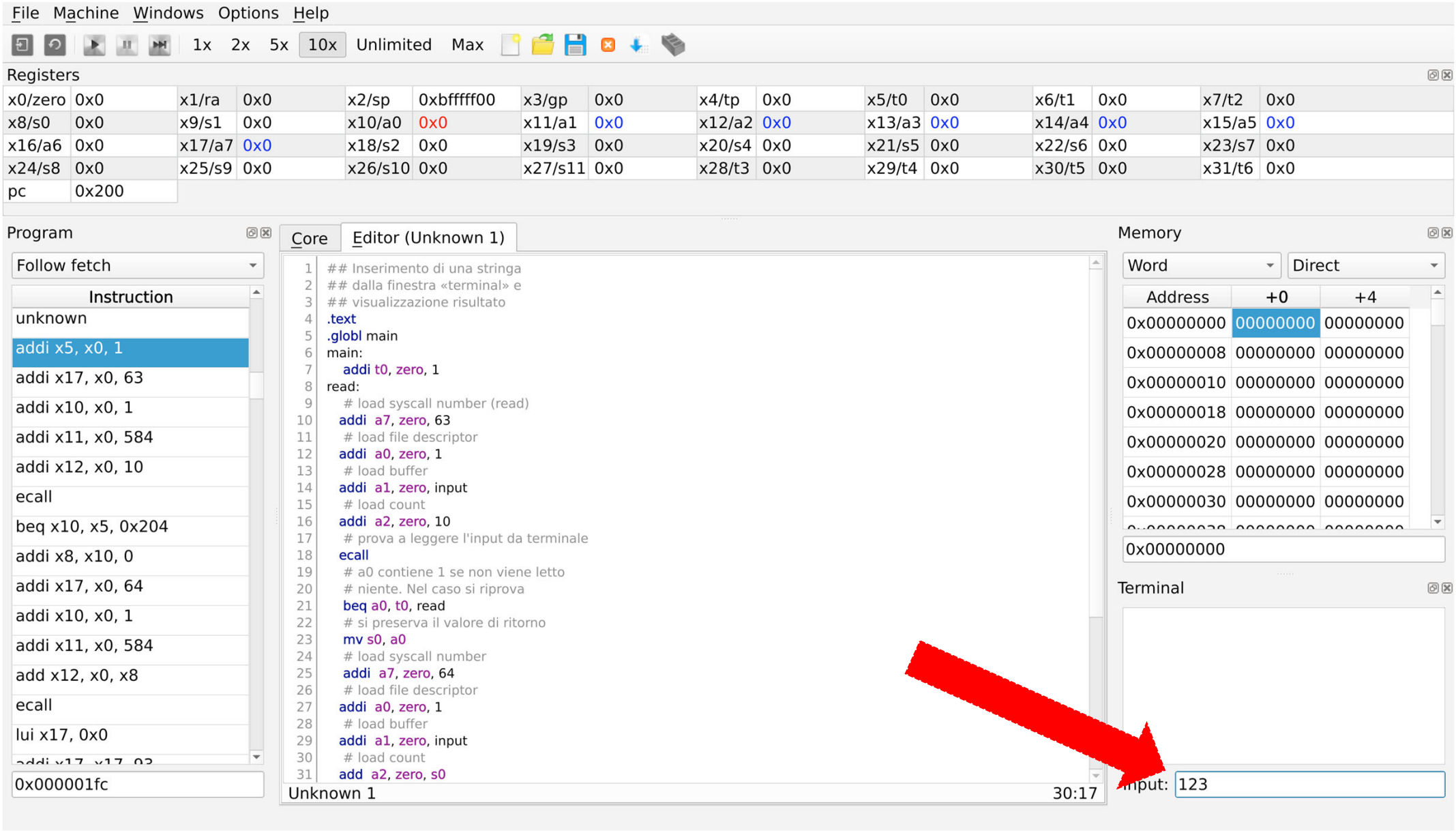
Esempio chiamate a sistema

```
## Inserimento di una stringa
## dalla finestra «terminal» e
## visualizzazione risultato
.text
.globl main
main:
    addi t0, zero, 1
read:
    # load syscall number (read)
    addi  a7, zero, 63
    # load file descriptor
    addi  a0, zero, 1
    # load buffer
    addi  a1, zero, input
    # load count
    addi  a2, zero, 10
    # prova a leggere l'input da terminale
    ecall
    # a0 contiene 1 se non viene letto
    # niente. Nel caso si riprova
    beq a0, t0, read
    # si preserva il valore di ritorno
    mv s0, a0
    # load syscall number
    addi  a7, zero, 64
    # load file descriptor
    addi  a0, zero, 1
    # load buffer
    addi  a1, zero, input
    # load count
    add  a2, zero, s0
    # stampa del contenuto del buffer su
    # terminale
    ecall

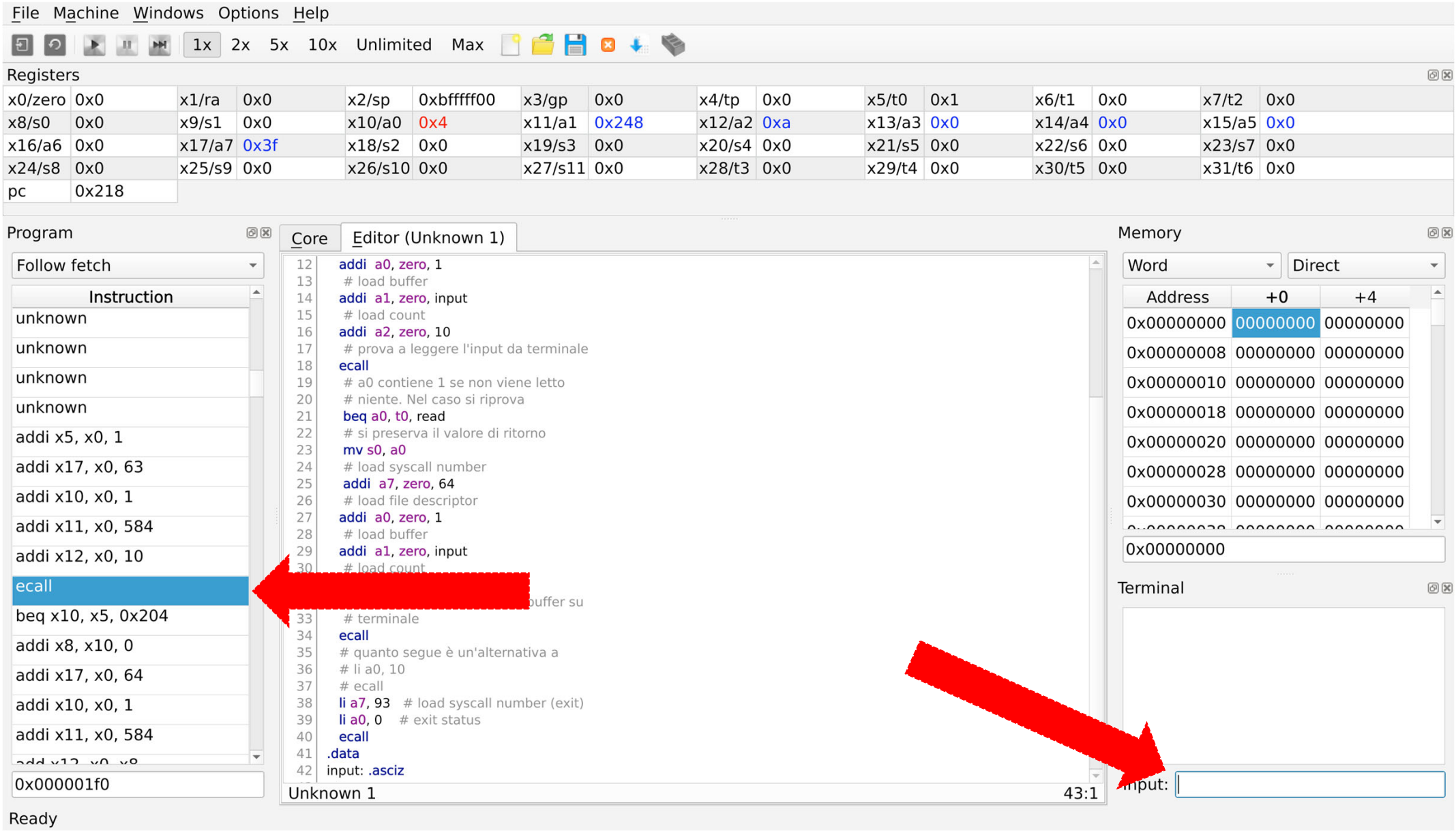
    # quanto segue è un'alternativa a
    # li a0, 10
    # ecall
    li a7, 93   # load syscall number (exit)
    li a0, 0    # exit status
    ecall

.data
input: .asciz
```

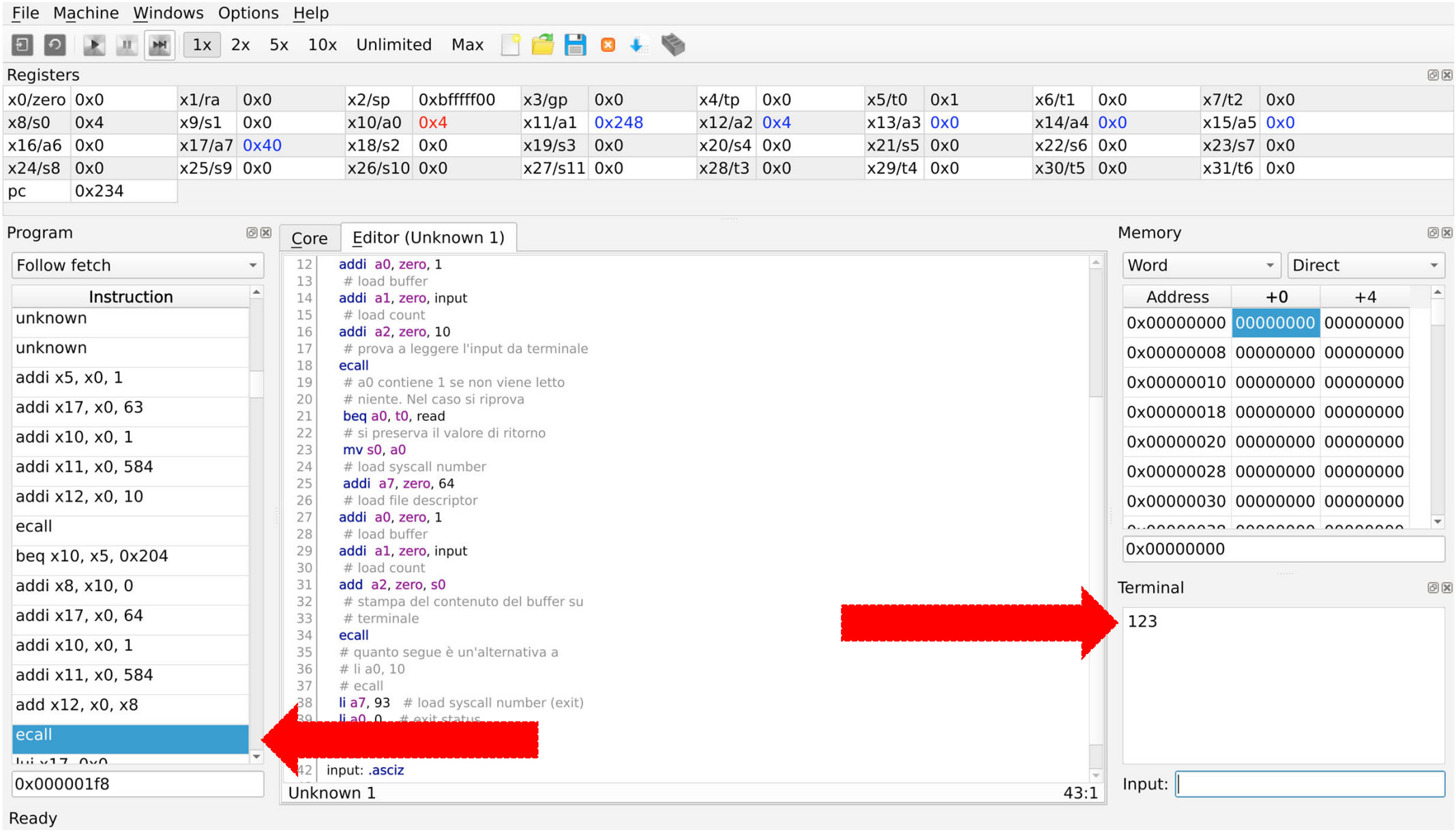
Esempio chiamate a sistema



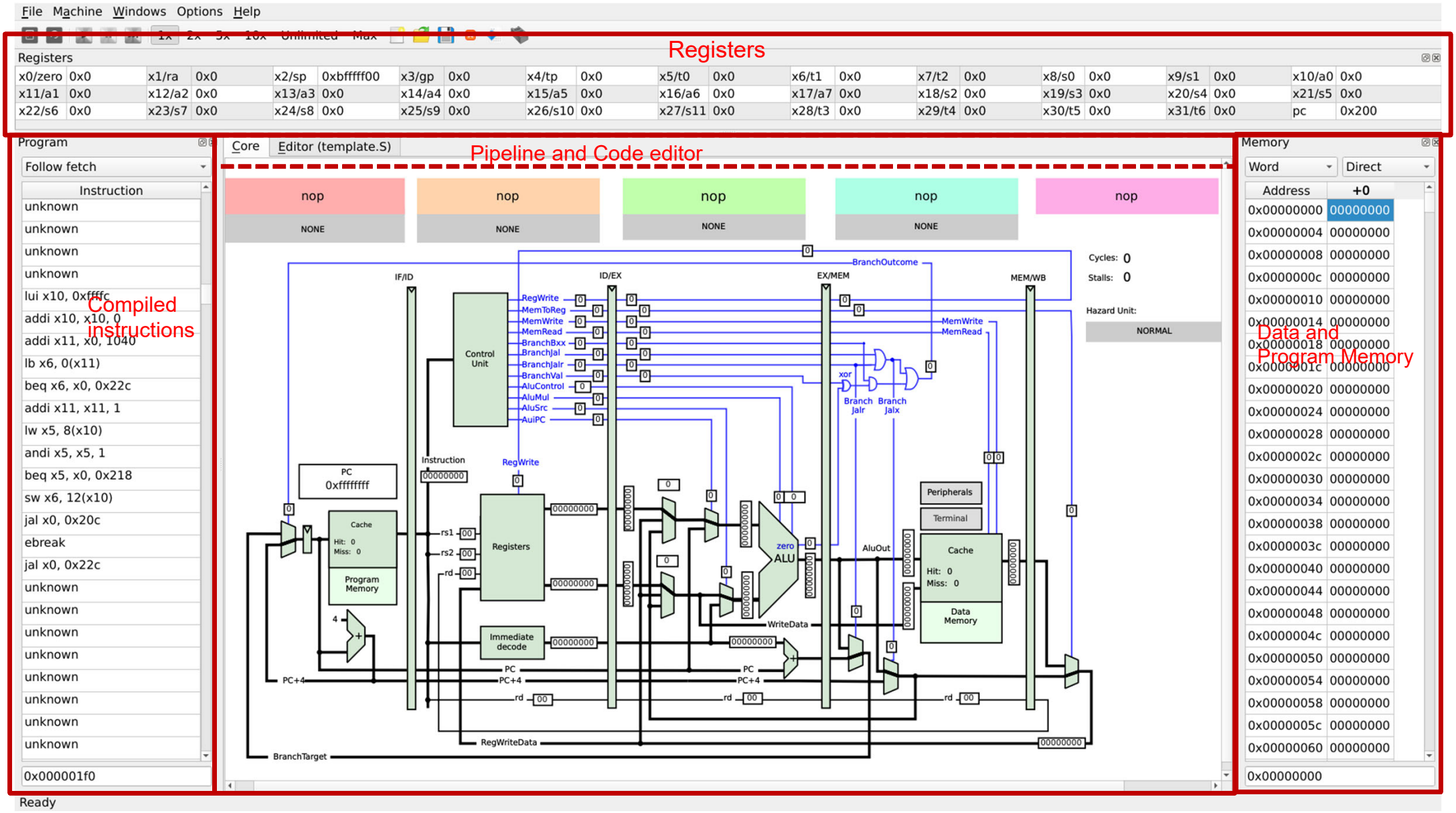
Esempio chiamate a sistema



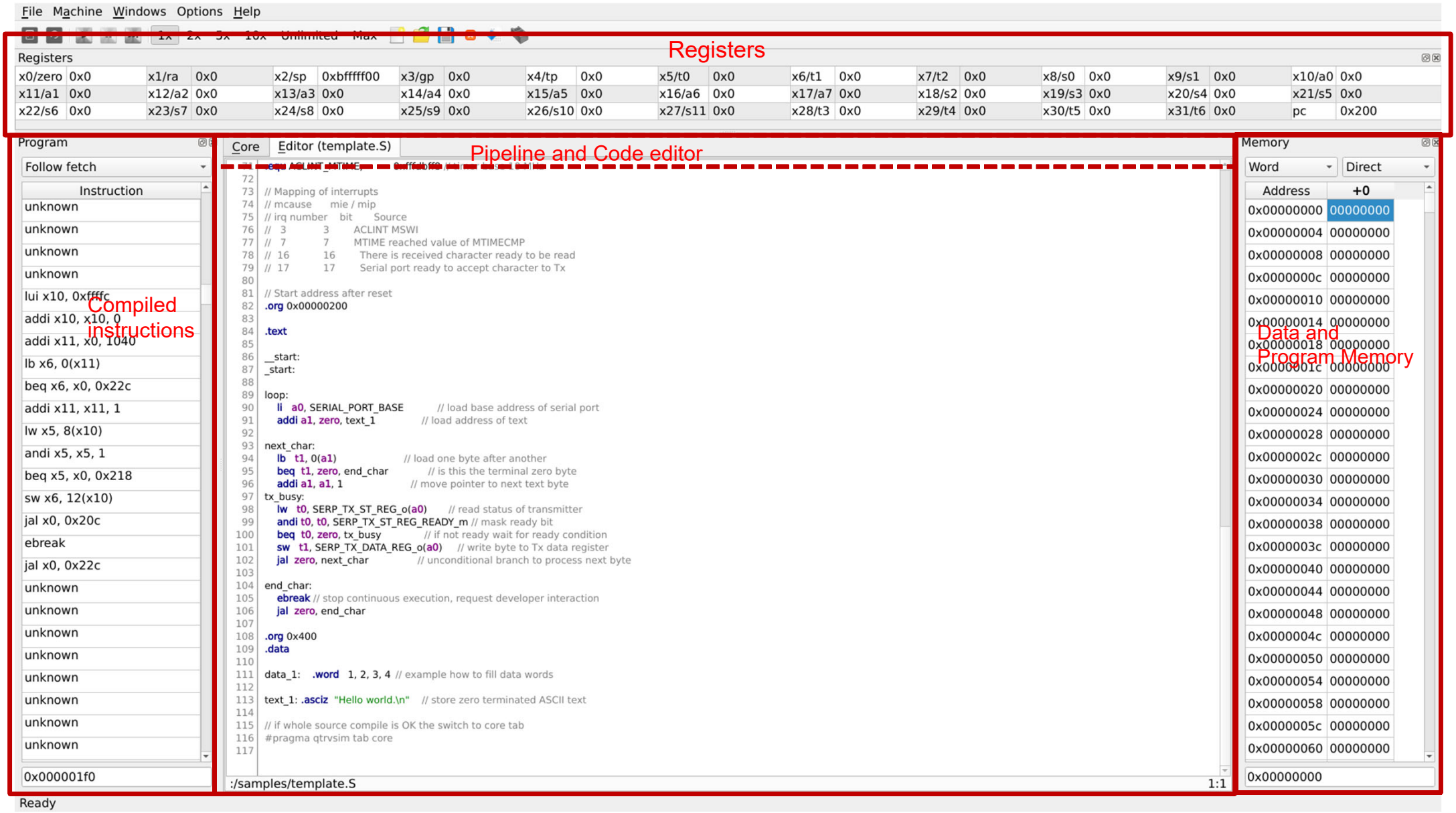
Esempio chiamate a sistema



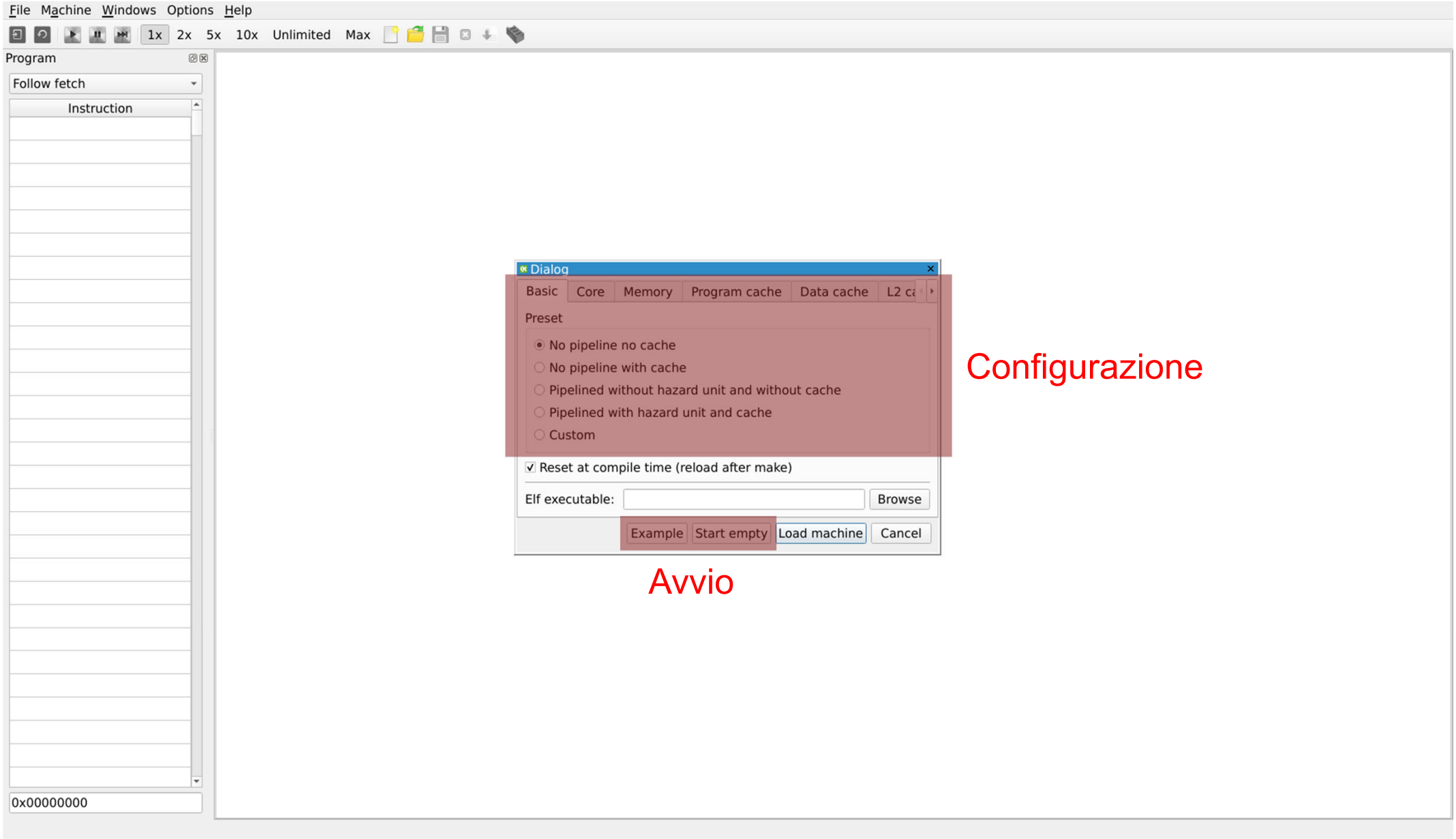
QtRvSim


File Machine Windows Options Help
1x 2x 5x 10x Unlimited Max
Registers
Registers
x0/zero 0x0 x1/ra 0x0 x2/sp 0xbfffff00 x3/gp 0x0 x4/tp 0x0 x5/t0 0x0 x6/t1 0x0 x7/t2 0x0 x8/s0 0x0 x9/s1 0x0 x10/a0 0x0
x11/a1 0x0 x12/a2 0x0 x13/a3 0x0 x14/a4 0x0 x15/a5 0x0 x16/a6 0x0 x17/a7 0x0 x18/s2 0x0 x19/s3 0x0 x20/s4 0x0 x21/s5 0x0
x22/s6 0x0 x23/s7 0x0 x24/s8 0x0 x25/s9 0x0 x26/s10 0x0 x27/s11 0x0 x28/t3 0x0 x29/t4 0x0 x30/t5 0x0 x31/t6 0x0 pc 0x200
Program
Follow fetch
Instruction
unknown
unknown
unknown
unknown
lui x10, 0xffffc
addi x10, x10, 0
addi x11, x0, 1040
lb x6, 0(x11)
beq x6, x0, 0x22c
addi x11, x11, 1
lw x5, 8(x10)
andi x5, x5, 1
beq x5, x0, 0x218
sw x6, 12(x10)
jal x0, 0x20c
ebreak
jal x0, 0x22c
unknown
unknown
unknown
unknown
unknown
unknown
unknown
unknown
0x000001f0
Compiled instructions
Core
Editor (template.S)
Pipeline and Code editor
nop
NONE
IF/ID
ID/EX
EX/MEM
MEM/WB
Cycles: 0
Stalls: 0
Hazard Unit:
NORMAL
Control Unit
RegWrite
MemToReg
MemWrite
MemRead
BranchBxx
BranchJal
BranchJalr
BranchVal
AluControl
AluMul
AluSrc
AuiPC
BranchOutcome
xor
Branch Jalr
Branch Jalx
PC
0xffffffff
Instruction
Cache
Hit: 0
Miss: 0
Program Memory
Registers
rs1
rs2
rd
Immediate decode
ALU
zero
AluOut
WriteData
Peripherals
Terminal
Data Memory
PC+4
RegWriteData
BranchTarget
Memory
Word
Direct
Address
+0
0x00000000 00000000
0x00000004 00000000
0x00000008 00000000
0x0000000c 00000000
0x00000010 00000000
0x00000014 00000000
0x00000018 00000000
0x0000001c 00000000
0x00000020 00000000
0x00000024 00000000
0x00000028 00000000
0x0000002c 00000000
0x00000030 00000000
0x00000034 00000000
0x00000038 00000000
0x0000003c 00000000
0x00000040 00000000
0x00000044 00000000
0x00000048 00000000
0x0000004c 00000000
0x00000050 00000000
0x00000054 00000000
0x00000058 00000000
0x0000005c 00000000
0x00000060 00000000
0x00000000
Data and Program Memory
Ready

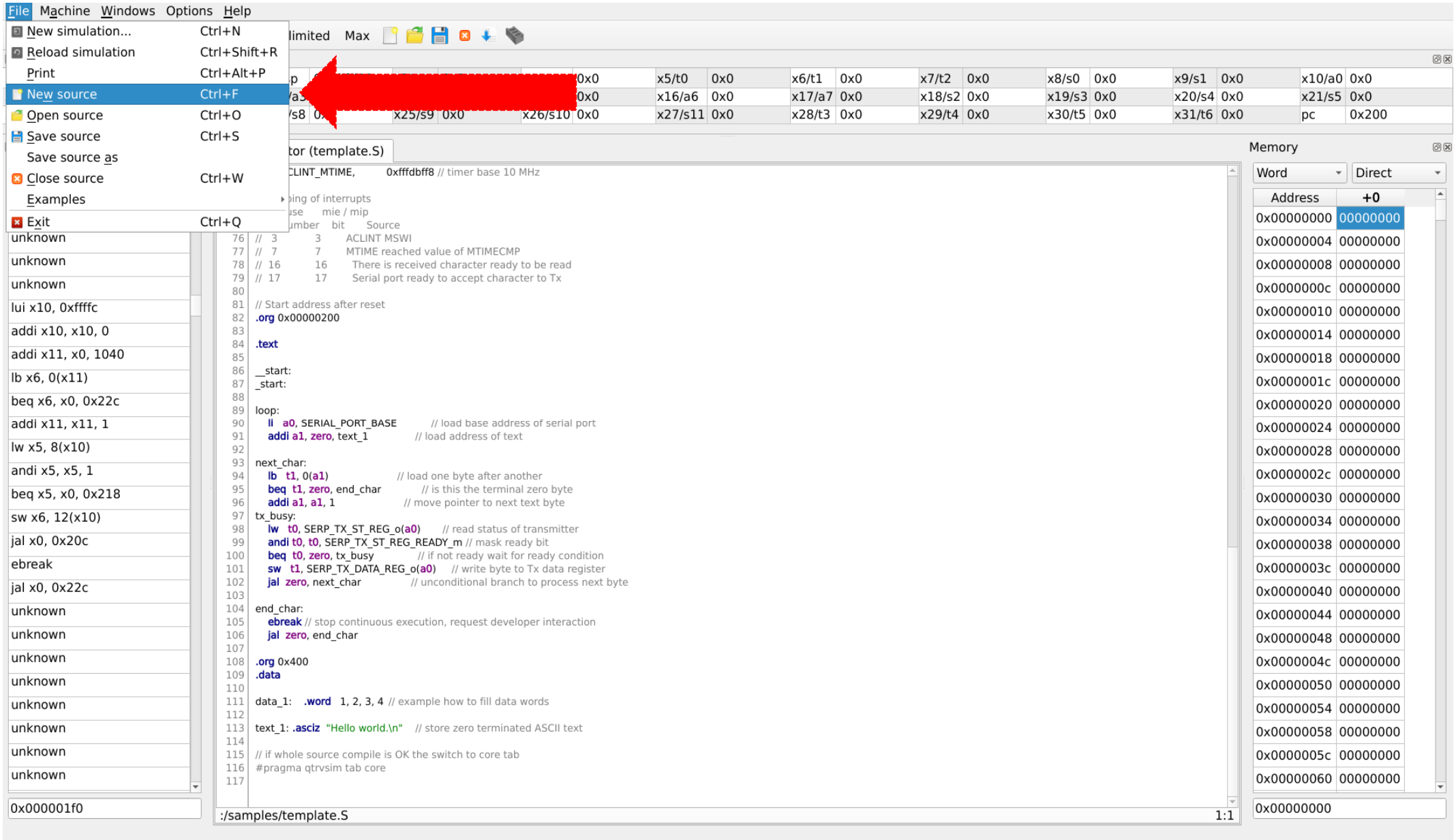
QtRvSim


File Machine Windows Options Help
1x 2x 5x 10x Unlimited Max
Registers
Registers
x0/zero 0x0 x1/ra 0x0 x2/sp 0xbfffff00 x3/gp 0x0 x4/tp 0x0 x5/t0 0x0 x6/t1 0x0 x7/t2 0x0 x8/s0 0x0 x9/s1 0x0 x10/a0 0x0
x11/a1 0x0 x12/a2 0x0 x13/a3 0x0 x14/a4 0x0 x15/a5 0x0 x16/a6 0x0 x17/a7 0x0 x18/s2 0x0 x19/s3 0x0 x20/s4 0x0 x21/s5 0x0
x22/s6 0x0 x23/s7 0x0 x24/s8 0x0 x25/s9 0x0 x26/s10 0x0 x27/s11 0x0 x28/t3 0x0 x29/t4 0x0 x30/t5 0x0 x31/t6 0x0 pc 0x200
Program
Follow fetch
Instruction
unknown
unknown
unknown
unknown
lui x10, 0xffffc
addi x10, x10, 0
addi x11, x0, 1040
lb x6, 0(x11)
beq x6, x0, 0x22c
addi x11, x11, 1
lw x5, 8(x10)
andi x5, x5, 1
beq x5, x0, 0x218
sw x6, 12(x10)
jal x0, 0x20c
ebreak
jal x0, 0x22c
unknown
unknown
unknown
unknown
unknown
unknown
unknown
unknown
0x000001f0
Compiled instructions
Core
Editor (template.S)
Pipeline and Code editor
72
73 // Mapping of interrupts
74 // mcause mie / mip
75 // irq number bit Source
76 // 3 3 ACLINT MSWI
77 // 7 7 MTIME reached value of MTIMECMP
78 // 16 16 There is received character ready to be read
79 // 17 17 Serial port ready to accept character to Tx
80
81 // Start address after reset
82 .org 0x00000200
83
84 .text
85
86 __start:
87 _start:
88
89 loop:
90 li a0, SERIAL_PORT_BASE // load base address of serial port
91 addi a1, zero, text_1 // load address of text
92
93 next_char:
94 lb t1, 0(a1) // load one byte after another
95 beq t1, zero, end_char // is this the terminal zero byte
96 addi a1, a1, 1 // move pointer to next text byte
97 tx_busy:
98 lw t0, SERP_TX_ST_REG_o(a0) // read status of transmitter
99 andi t0, t0, SERP_TX_ST_REG_READY_m // mask ready bit
100 beq t0, zero, tx_busy // if not ready wait for ready condition
101 sw t1, SERP_TX_DATA_REG_o(a0) // write byte to Tx data register
102 jal zero, next_char // unconditional branch to process next byte
103
104 end_char:
105 ebreak // stop continuous execution, request developer interaction
106 jal zero, end_char
107
108 .org 0x400
109 .data
110
111 data_1: .word 1, 2, 3, 4 // example how to fill data words
112
113 text_1: .asciz "Hello world.\n" // store zero terminated ASCII text
114
115 // if whole source compile is OK the switch to core tab
116 #pragma qtrvsim tab core
117
:/samples/template.S
1:1
Memory
Word
Direct
Address +0
0x00000000 00000000
0x00000004 00000000
0x00000008 00000000
0x0000000c 00000000
0x00000010 00000000
0x00000014 00000000
0x00000018 00000000
0x0000001c 00000000
0x00000020 00000000
0x00000024 00000000
0x00000028 00000000
0x0000002c 00000000
0x00000030 00000000
0x00000034 00000000
0x00000038 00000000
0x0000003c 00000000
0x00000040 00000000
0x00000044 00000000
0x00000048 00000000
0x0000004c 00000000
0x00000050 00000000
0x00000054 00000000
0x00000058 00000000
0x0000005c 00000000
0x00000060 00000000
0x00000000
Data and Program Memory
Ready

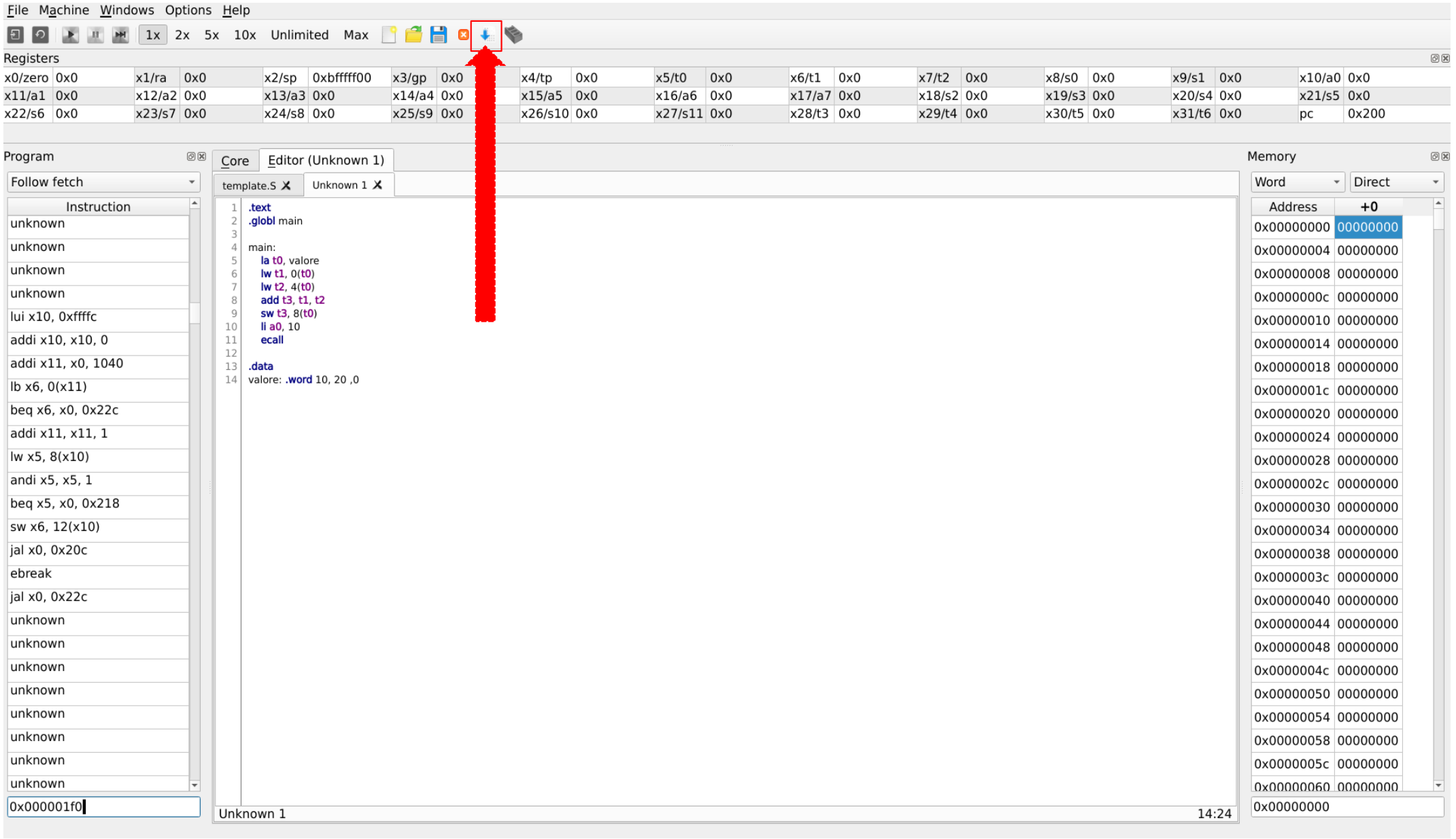
Avvio: configurare il tipo di simulazione



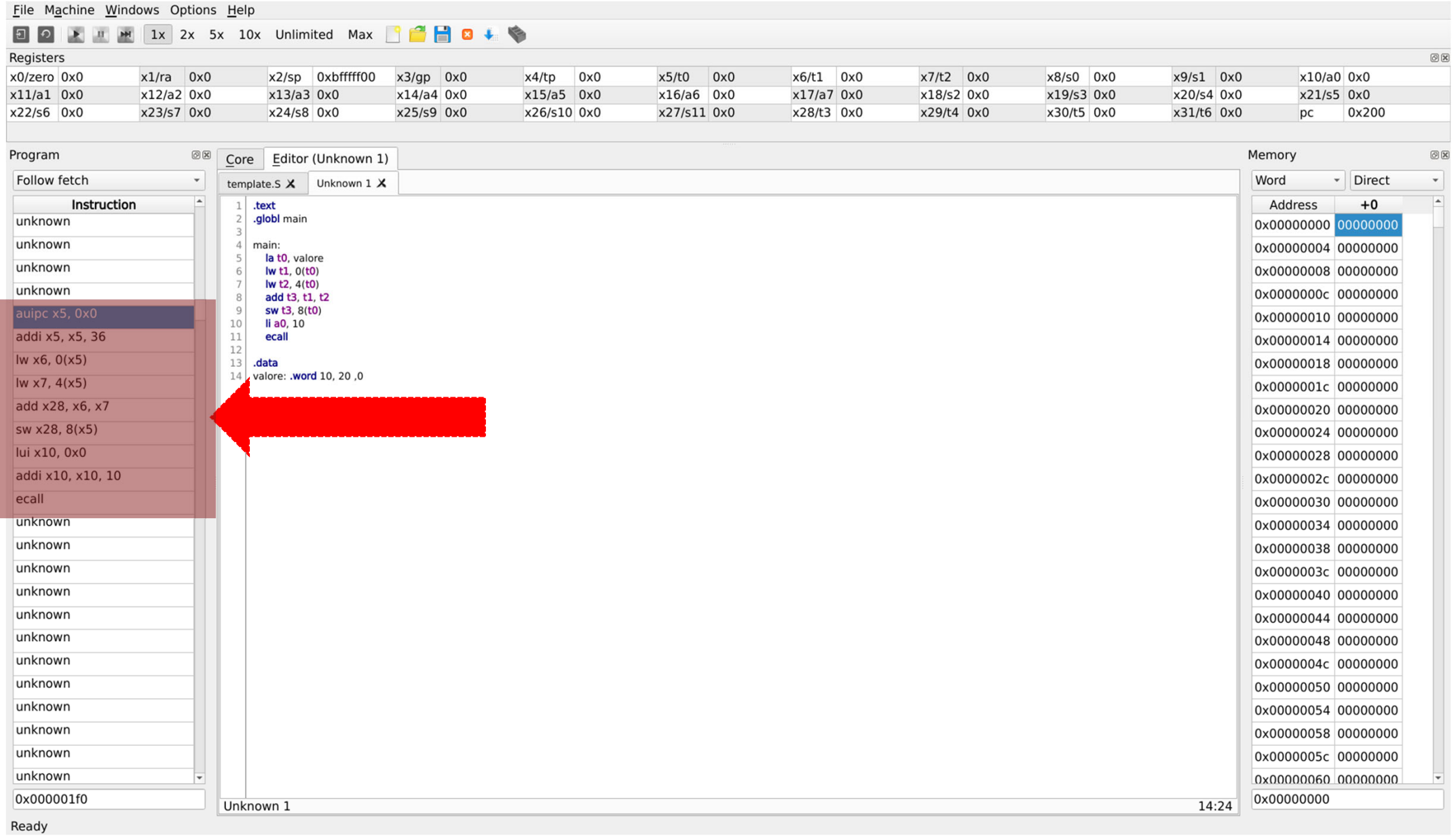
Nuovo codice



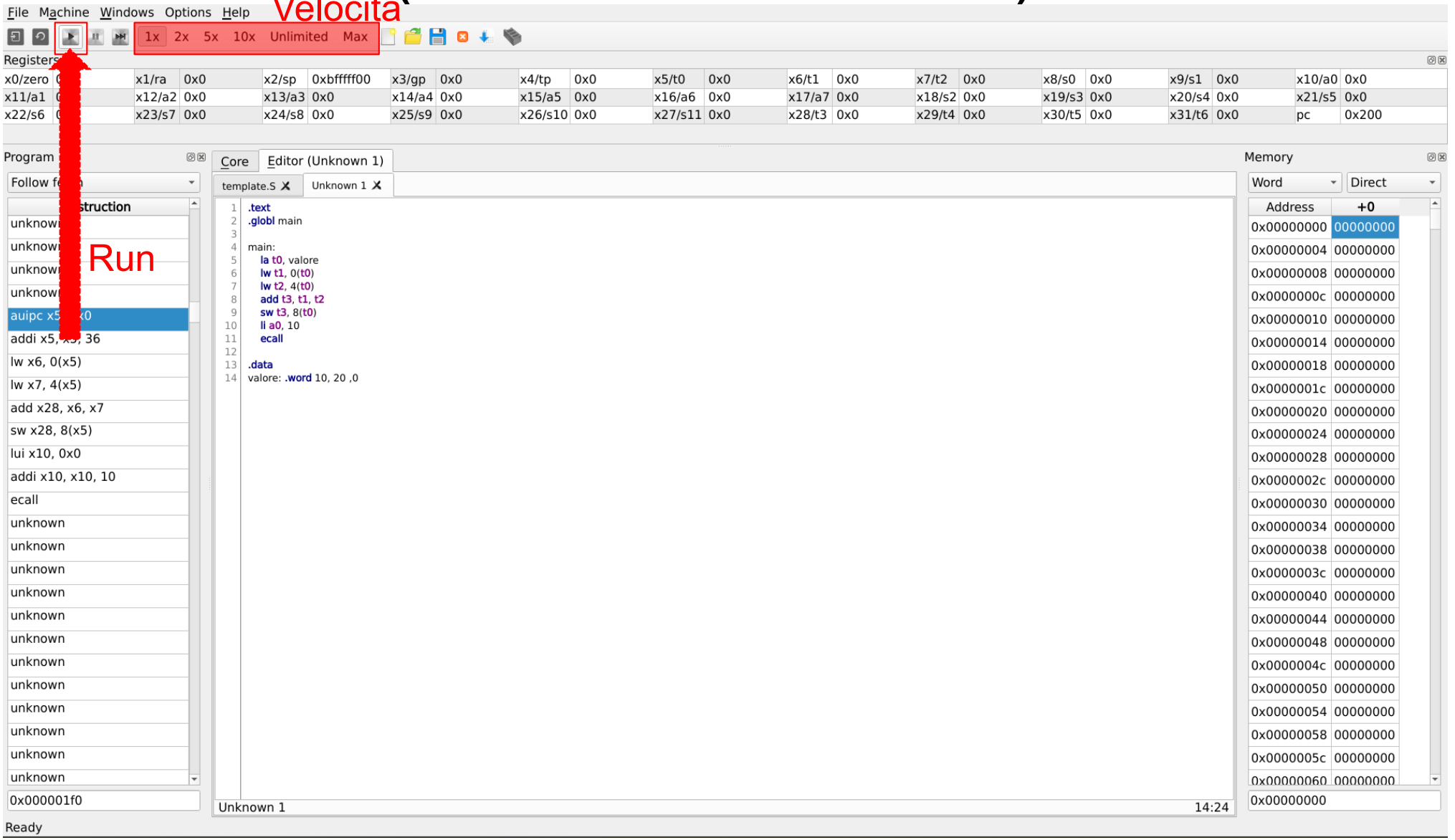
Compilazione sorgente ed aggiornamento memoria



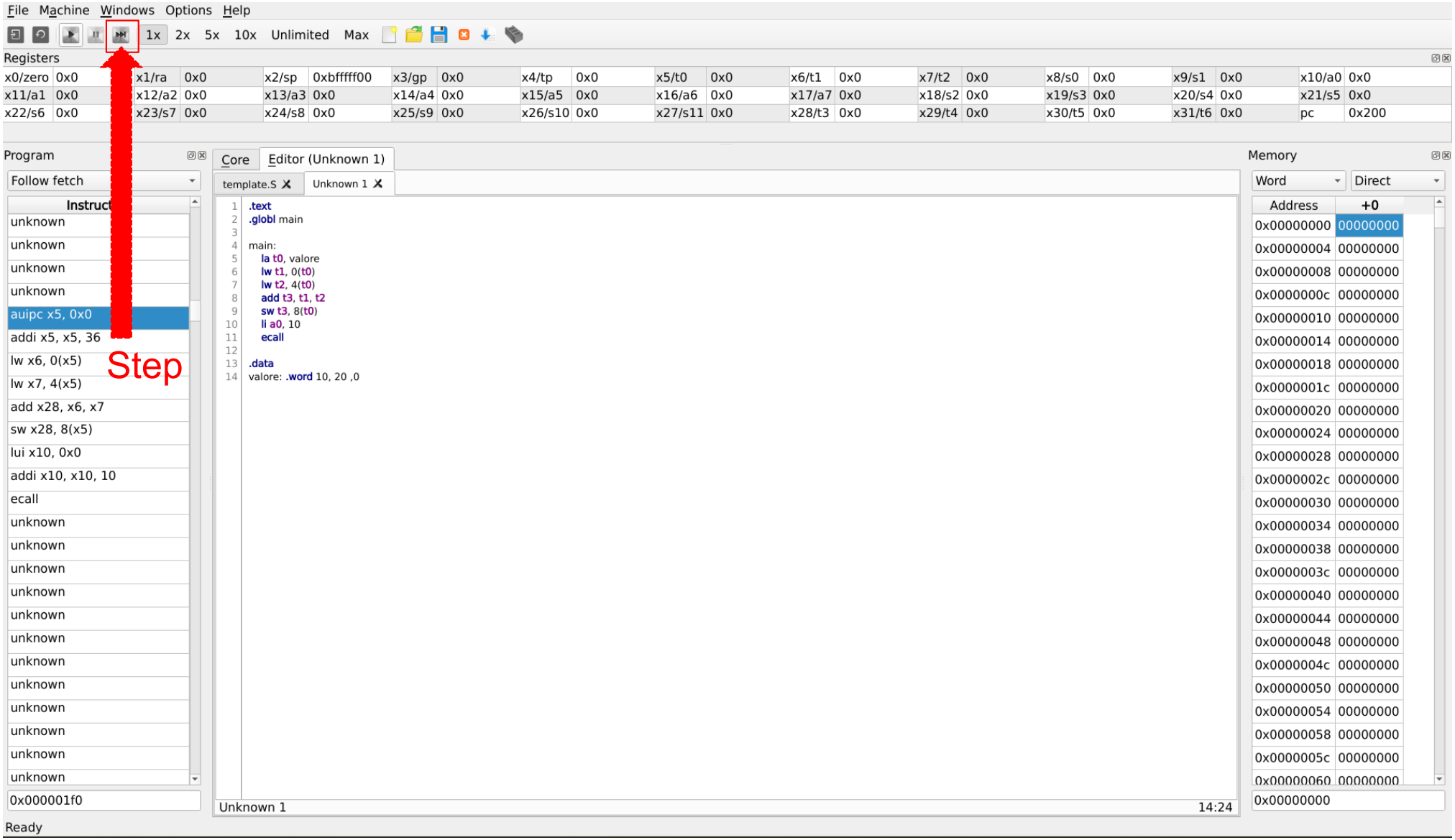
Compilazione sorgente ed aggiornamento memoria



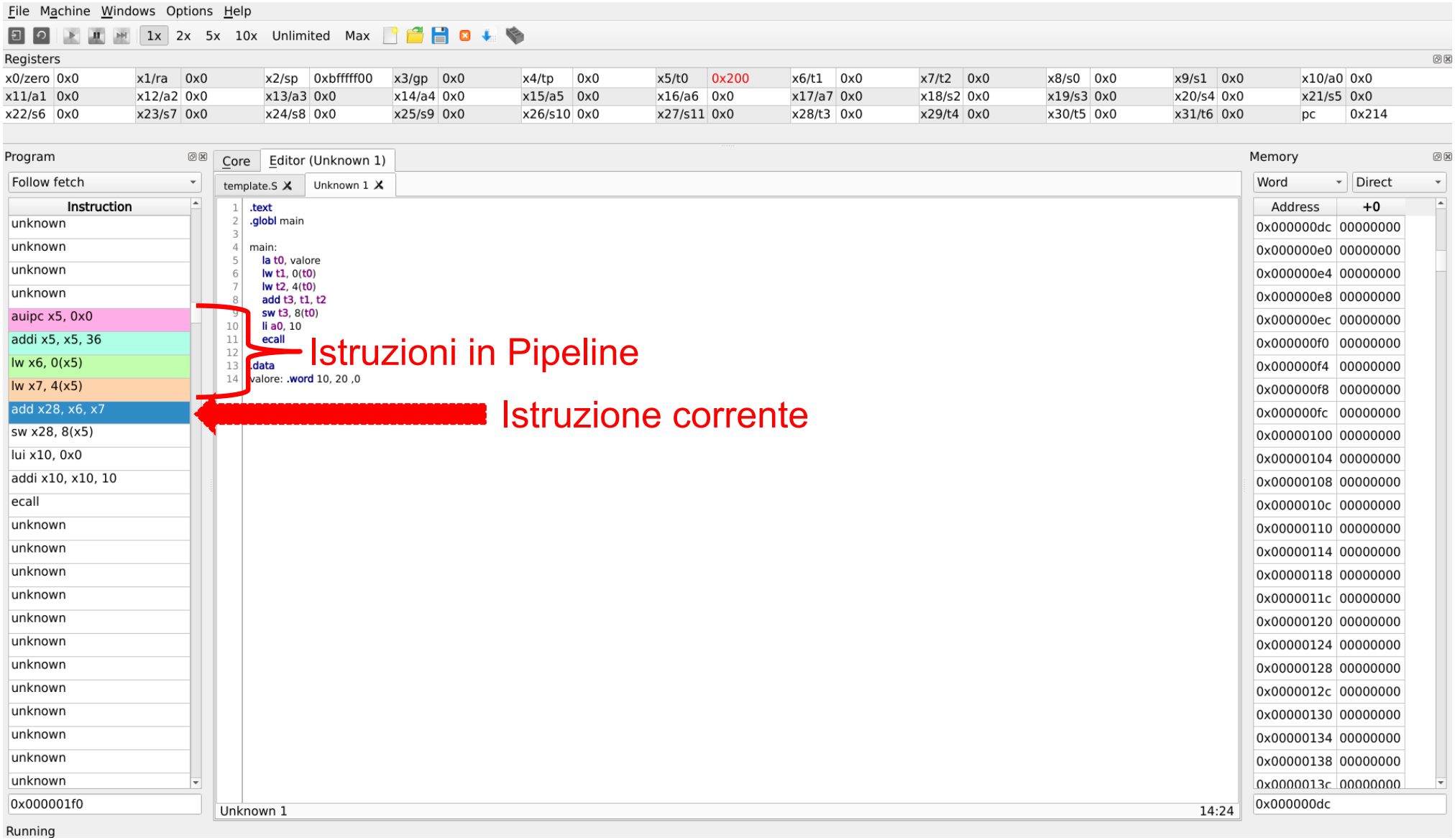
Esecuzione step-by-step automatica (con diverse velocità)



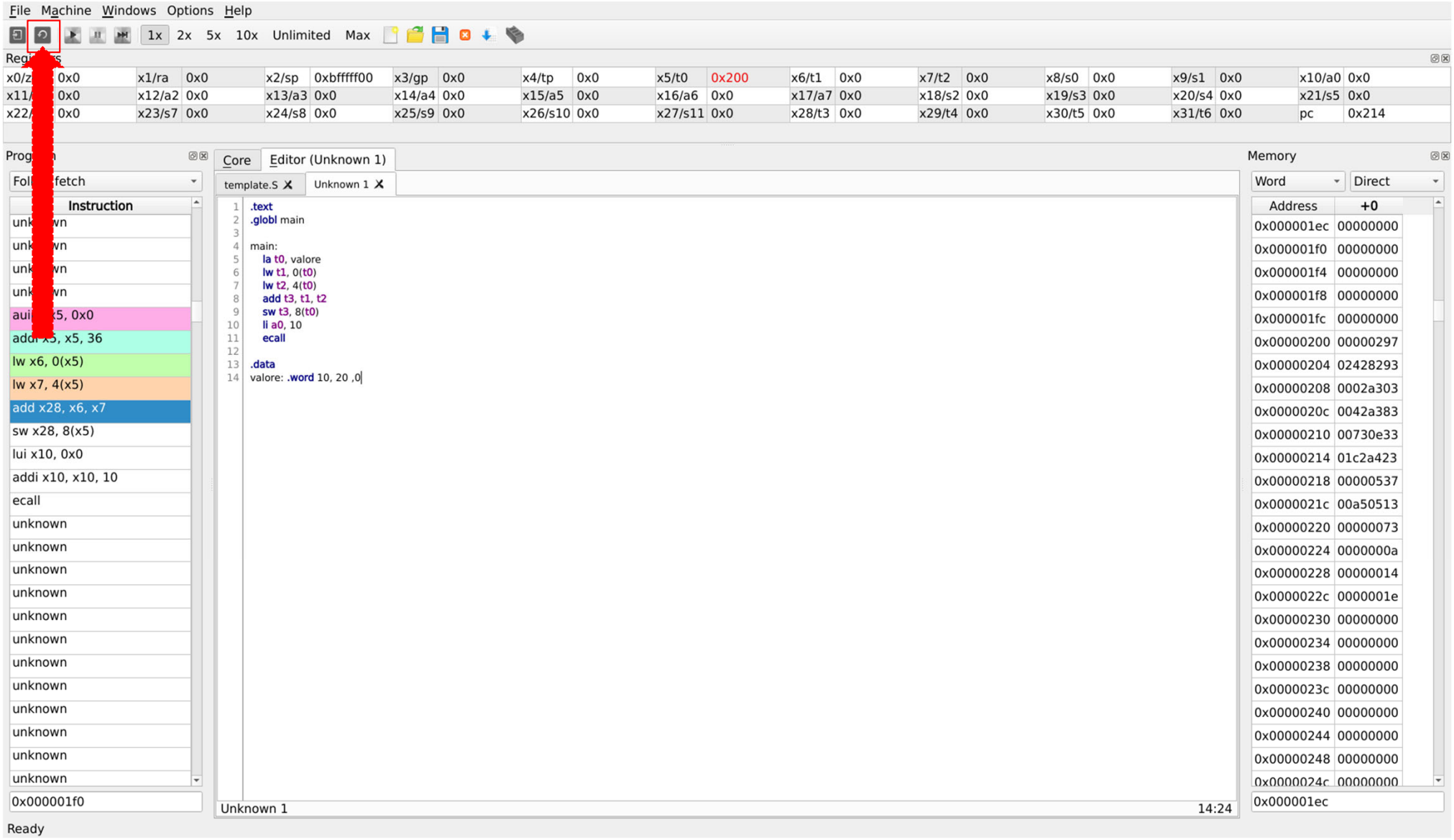
Esecuzione step-by-step manuale



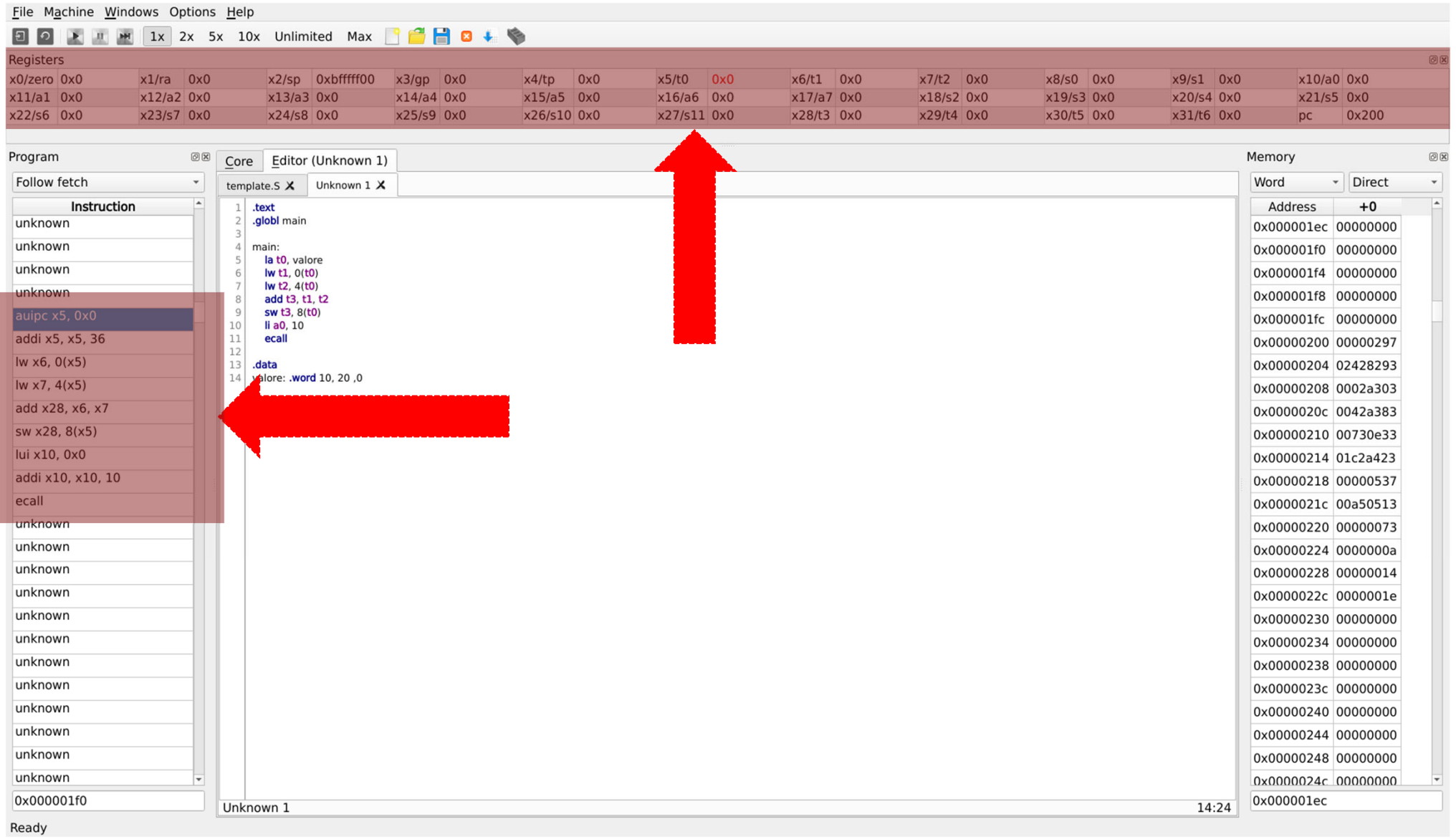
Esecuzione


File Machine Windows Options Help
1x 2x 5x 10x Unlimited Max
Registers
x0/zero 0x0 x1/ra 0x0 x2/sp 0xbfffff00 x3/gp 0x0 x4/tp 0x0 x5/t0 0x200 x6/t1 0x0 x7/t2 0x0 x8/s0 0x0 x9/s1 0x0 x10/a0 0x0
x11/a1 0x0 x12/a2 0x0 x13/a3 0x0 x14/a4 0x0 x15/a5 0x0 x16/a6 0x0 x17/a7 0x0 x18/s2 0x0 x19/s3 0x0 x20/s4 0x0 x21/s5 0x0
x22/s6 0x0 x23/s7 0x0 x24/s8 0x0 x25/s9 0x0 x26/s10 0x0 x27/s11 0x0 x28/t3 0x0 x29/t4 0x0 x30/t5 0x0 x31/t6 0x0 pc 0x214
Program
Follow fetch
Instruction
unknown
unknown
unknown
unknown
auipc x5, 0x0
addi x5, x5, 36
lw x6, 0(x5)
lw x7, 4(x5)
add x28, x6, x7
sw x28, 8(x5)
lui x10, 0x0
addi x10, x10, 10
ecall
unknown
unknown
unknown
unknown
unknown
unknown
unknown
unknown
unknown
unknown
unknown
0x000001f0
Core
Editor (Unknown 1)
template.S
Unknown 1
1 .text
2 .globl main
3
4 main:
5 la t0, valore
6 lw t1, 0(t0)
7 lw t2, 4(t0)
8 add t3, t1, t2
9 sw t3, 8(t0)
10 li a0, 10
11 ecall
12
13 .data
14 valore: .word 10, 20 ,0
Istruzioni in Pipeline
Istruzione corrente
Unknown 1
14:24
Memory
Word
Direct
Address +0
0x000000dc 00000000
0x000000e0 00000000
0x000000e4 00000000
0x000000e8 00000000
0x000000ec 00000000
0x000000f0 00000000
0x000000f4 00000000
0x000000f8 00000000
0x000000fc 00000000
0x00000100 00000000
0x00000104 00000000
0x00000108 00000000
0x0000010c 00000000
0x00000110 00000000
0x00000114 00000000
0x00000118 00000000
0x0000011c 00000000
0x00000120 00000000
0x00000124 00000000
0x00000128 00000000
0x0000012c 00000000
0x00000130 00000000
0x00000134 00000000
0x00000138 00000000
0x0000013c 00000000
0x000000dc
Running

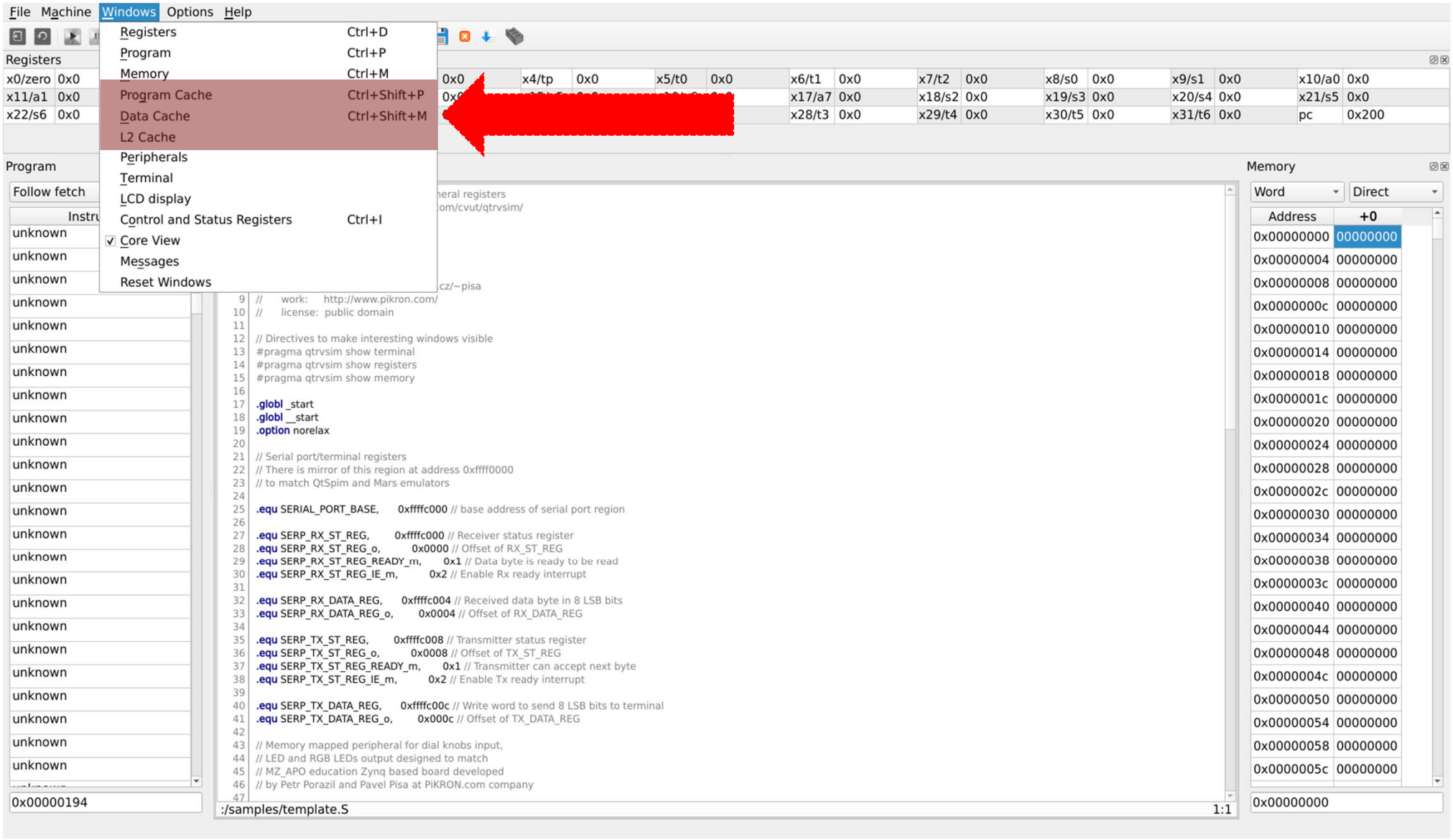
Reset simulazione



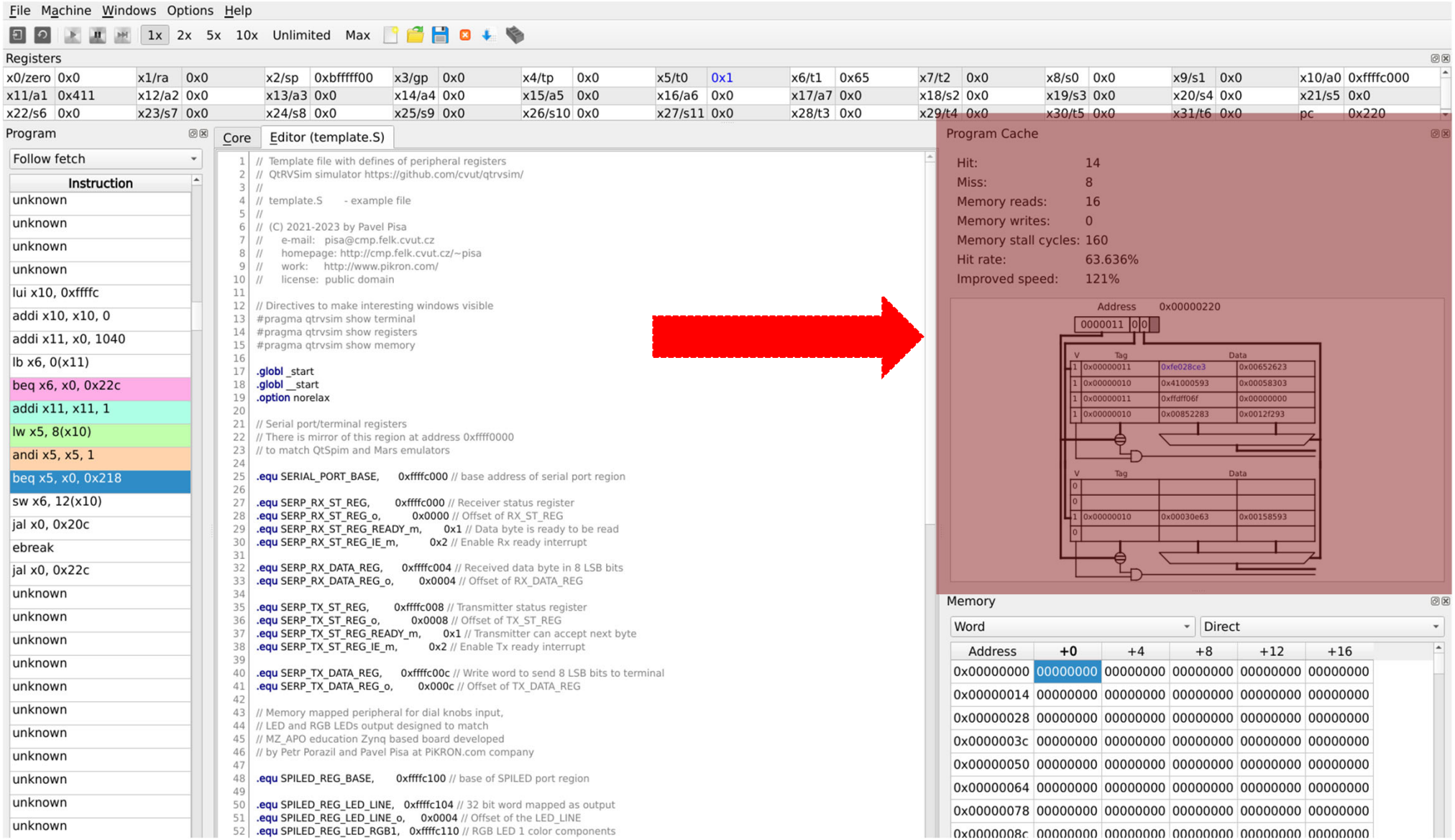
Reset simulazione



Visualizzare le cache



Visualizzare le cache



Pseudoistruzioni

• **mv t0, s0**	addi t0, s0, 0
• **li rs, imm**	lui rs, imm[31:12] addi rs, rs, imm[11:0]
• **beqz rs, offset**	beq rs, x0, offset
• **bnez rs, offset**	bne rs, x0, offset
• **j offset**	jal x0, offset
• **jr rs**	jalr x0, 0(rs)
• **ret**	jalr x0, 0(x1)
• **bqt rs, rt, offset**	blt rt, rs, offset
• **la rd, symbol**	Load address (of symbol) in rd
• **…**	

Esempio: prodotto vettori

```
#include <stdio.h>
int main(int argc, char** argv) {
   int vectorA[5] = {1,2,3,4,5};
   int vectorB[5] = {2,4,6,8,10};
   int result = 0;
   int i=0;
   while (i<5) {
      result += vectorA[i]*vectorB[i];
      i+=1;
   }
   printf( “result %d\n” ,result);
}
```

esplicitare il codice

```
int main(int argc, char** argv) {
   int vectorA[5] = {1,2,3,4,5};
   int vectorB[5] = {2,4,6,8,10};
   int result = 0;
   int i=0;
   int valueA = 0;
   int valueB = 0;
   while (i<5) {
      valueA = vectorA[i];
      valueB = vectorB[i];
      result += valueA*valueB;
      i+=1;
   }
}
```

```
int main(int argc, char** argv) {
   int vectorA[5] = {1,2,3,4,5};
   int vectorB[5] = {2,4,6,8,10};
   int result = 0;
   int i=0;
   int valueA = 0;
   int valueB = 0;
   bool condition = true;
   while (condition) {
      valueA = vectorA[i];
      valueB = vectorB[i];
      result += valueA*valueB;
      i+=1;
      condition = (i>=5) ? false : true;
   }
}
```

separare salti
da calcolo condizione

```
int main(int argc, char** argv) {
   int vectorA[5] = {1,2,3,4,5};
   int vectorB[5] = {2,4,6,8,10};
   int result = 0;
   int i=0;
   int valueA = 0;
   int valueB = 0;
   while (i<5) {
      valueA = vectorA[i];
      valueB = vectorB[i];
      result += valueA*valueB;
      i+=1;
   }
}
```

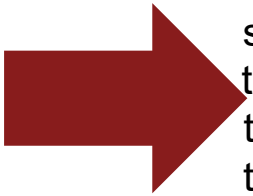
```
int main(int argc, char** argv) {
   int vectorA[5] = {1,2,3,4,5};
   int vectorB[5] = {2,4,6,8,10};
   int result = 0;
   int i=0;
   int valueA = 0;
   int valueB = 0;
   bool condition = true;
   while (condition) {
      valueA = vectorA[i];
      valueB = vectorB[i];
      result += valueA*valueB;
      i+=1;
      condition = (i>=5) ? false : true;
   }
}
```

```
int main(int argc, char** argv) {
   int vectorA[5] = {1,2,3,4,5};
   int vectorB[5] = {2,4,6,8,10};
   int result = 0;
   int intermidiateResult = 0;
   int i=0;
   int* addressA = vectorA;
   int* addressB = vectorB;
   int valueA = 0;
   int valueB = 0;
   bool condition = true;
   while (condition) {
      valueA = *(addressA);
      valueB = *(addressB);
      intermidiateResult = valueA*valueB;
      result = result + intermidiateResult;
      i+=1;
      addressA+=1;
      addressB+=1;
      condition = (i>=5) ? false : true;
   }
}
```

Esplicitare le singole operazioni

Esplicitare gli accessi in memoria

```
int main(int argc, char** argv) {
  int vectorA[5] = {1,2,3,4,5};
  int vectorB[5] = {2,4,6,8,10};
  int result = 0;
  int intermidiateResult = 0;
  int i=0;
  int* addressA = vectorA;
  int* addressB = vectorB;
  int valueA = 0;
  int valueB = 0;
  bool condition = true;
  while (condition) {
    valueA = *(addressA);
    valueB = *(addressB);
    intermidiateResult = valueA*valueB;
    result += intermidiateResult;
    i+=1;
    addressA+=1;
    addressB+=1;
    condition = (i>=5) ? false : true;
  }
}
```



Associare variabili e registri

s0
s1
s2
t6
s3
t2
t3
t4
t5
t0

Annotare il codice

s0 stores the address of vectorA
s1 stores the address of vectorB
s2 stores the final result (initialized to x0)
s3 counter i
t0 condition
t1 internal flag used to compare to 1
t2 stores the address of vectorA[i]
t3 stores the address of vectorB[i]
t4 stores the value of vectorA[i]
t5 stores the value of vectorB[i]
t6 stores the intermidiate addition of t4 and t5

```
# ====================================
# Description:  perform dot product of 2 vectors
# Test:
# A = [1,2,3,4,5] = [0x1,0x2,0x3,0x4,0x5]
# B = [2,4,6,8,10] = [0x2,0x4,0x6,0x8,0xA]
# Expected result
# R = A.B = 2+8+18+32+50 = 110 = 0x6E
# ====================================
# Your annotated registers

# ========== Data Segment
.data
#your data will come here

# ========== Code Segment
.text
.globl  main
main:
# your code will come here

EXIT:
li a0,10
ecall

# End of file
```

s0 stores the address of vectorA
s1 stores the address of vectorB
s2 stores the final result (initialized to $zero)
s3 counter i
t0 condition
t1 internal flag used to compare to 1
t2 stores the address of vectorA[i]
t3 stores the address of vectorB[i]
t4 stores the value of vectorA[i]
t5 stores the value of vectorB[i]
t6 stores the intermediate addition of t4 and t5

```
vectorA:  .word 1,2,3,4,5
vectorB:  .word 2,4,6,8,10
```

Convertire codice C in assembler RISC-V

```
int main(int argc, char** argv) {
   int vectorA[5] = {1,2,3,4,5};
   int vectorB[5] = {2,4,6,8,10};
   int result = 0;
   int intermidiateResult = 0;
   int i=0;
   int* addressA = vectorA;
   int* addressB = vectorB;
   int valueA = 0;
   int valueB = 0;
   bool condition = true;
   while (condition) {
      valueA = *(addressA);
      valueB = *(addressB);
      intermidiateResult = valueA*valueB;
      result += intermidiateResult;
      i+=1;
      addressA+=1;
      addressB+=1;
      condition = (i>=5) ? false : true;
   }
}
```

```
main:
la s0, vectorA       # [pseudo] puts address of vectorA into s0
la s1, vectorB       # [pseudp] puts address of vectorB into s1
addi s2, x0, 0  # initialized the result to zero
addi s3, x0, 0  # i=0
addi t1, x0, 1   # t1=1
addi t2, s0, 0      # t2 stores the address of a[0]
addi t3, s1, 0      # t3 stores the address of b[0]

LOOP:
slti t0, s3, 5          # $t0=1 if i < 5
bne t0, t1, EXIT  # if i >= 5, exit from the loop
lw t4, 0(t2)          # load a[i] to t4
lw t5, 0(t3)          # load b[i] to t5
mul t6, t5, t4           # b[i]*a[i]

add s2,s2,t6
addi s3, s3, 1      # i=i+1
addi t2, t2, 4       # increment address of a[] by 4 bytes, 1 ptr.
addi t3, t3, 4       # increment address of b[] by 4 bytes, 1 ptr.
j LOOP
EXIT:
```

```
# ====================================
# Description:  perform dot product of 2 vectors
# Test:
# A = [1,2,3,4,5] = [0x1,0x2,0x3,0x4,0x5]
# B = [2,4,6,8,10] = [0x2,0x4,0x6,0x8,0xA]
# Expected result
# R = A.B = 2+8+18+32+50 = 110 = 0x6E
# ====================================
# s0 stores the address of vectorA
# s1 stores the address of vectorB
# s2 stores the final result (initialized to $zero)
# s3 counter i
# t0 condition
# t1 internal flag used to compare to 1
# t2 stores the address of vectorA[i]
# t3 stores the address of vectorB[i]
# t4 stores the value of vectorA[i]
# t5 stores the value of vectorB[i]
# t6 stores the intermediate addition of t4 and t5

# ========== Data Segment
.data
vectorA:  .word 1, 2, 3, 4, 5
vectorB:  .word 2, 4, 6, 8, 10

# ========== Code Segment
.text
.globl  main

main:
la s0, vectorA        # [pseudo] puts the address of vectorA into s0
la s1, vectorB        # [pseudp] puts the address of vectorB into s1
addi s2, x0, 0   # initialized the result to zero
addi s3, x0, 0    # i=0
addi t1, x0, 1    # t1=1
addi t2, s0, 0       # t2 stores the address of a[0]
addi t3, s1, 0       # t3 stores the address of b[0]

LOOP:
slti t0, s3, 5          # t0=1 if i < 5
bne t0, t1, EXIT    # if i >= 5, exit from the loop
lw t4, 0(t2)          # load a[i] to t4
lw t5, 0(t3)          # load b[i] to t5
mul t6, t5, t4             # b[i]*a[i]

add s2,s2,t6
addi s3, s3, 1        # i=i+1
addi t2, t2, 4        # increment address of a[] by 4 bytes, 1 ptr.
addi t3, t3, 4        # increment address of b[] by 4 bytes, 1 ptr.
j LOOP

EXIT:
li a7,93
ecall
# End of file
```